AI-Assisted Development Documentation

Learning without Forgetting (LwF) Reproduction Project

Nir Moyal
Dan Feldman

Sami Shamoon College of Engineering (SCE)

This document presents selected and condensed AI-assisted interactions that shaped the project. It focuses on how the chat was used to understand the paper, design the experiments, identify implementation mistakes, resolve runtime problems, interpret the results, and prepare the final submission.

June 2026

Contents

1 Purpose of This Documentation

The course requirements emphasize the development process, including the use of AI tools during the project. This document therefore records representative interactions with an AI assistant and explains how these interactions influenced the technical decisions made during the reproduction of the paper *Learning without Forgetting* by Li and Hoiem.

The goal is not to reproduce every chat message. Instead, the document highlights the conversations that had a meaningful effect on the project. The excerpts are condensed for readability while preserving the technical content and the sequence of decisions.

The AI assistant was used as:

- a paper-reading and explanation tool,
- an implementation planning assistant,
- a debugging partner,
- a reviewer of experimental consistency,
- a repository and documentation reviewer,
- and a guide for interpreting the final results.

All model training, data preparation, notebook execution, result collection, and repository submission were performed by the students.

2 Project Context

The project investigates **catastrophic forgetting**: the loss of previously learned capabilities when a neural network is adapted to a new task.

The reproduced setting is:

- **Old task:** ImageNet classification with 1,000 classes.
- **New task 1:** CUB-200-2011 bird classification with 200 classes.
- **New task 2:** MIT Indoor Scenes classification with 67 classes.

Three methods were compared:

1. **Fine-Tuning**
2. **Feature Extraction**
3. **Learning without Forgetting (LwF)**

The central idea of LwF is to learn a new task while preserving the responses of an original model, without using the original training data. The total loss is:

$$L = L_{\text{new}} + \lambda_{\text{old}} L_{\text{old}}$$

where L_{new} is the new-task classification loss and L_{old} is a knowledge-distillation loss that encourages the student model to preserve the original ImageNet behavior.

3 Development Timeline

Phase	Main Objective	Representative AI Contribution
Paper study	Understand catastrophic forgetting, LwF, temperature, and λ_{old}	Explained the old/new task setting and the role of distillation targets.
Dataset preparation	Load CUB and Scenes correctly	Guided metadata merging, official train/test split handling, and PyTorch Dataset construction.
Baseline implementation	Implement Fine-Tuning and Feature Extraction	Clarified what is frozen, what is trainable, and how old-task accuracy must be measured.
LwF implementation	Build a teacher-student dual-head model	Identified the required ImageNet and new-task heads and the correct use of teacher logits.
Debugging	Investigate suspicious results	Helped inspect model loading, head selection, frozen layers, checkpoint selection, and evaluation functions.
Protocol definition	Use one clear protocol for CUB and Scenes	Specified AlexNet for both experiments, one warm-up epoch, five main epochs, one fixed seed, and the same 80-batch ImageNet evaluation protocol.
Runtime troubleshooting	Resolve stalled notebook execution	Diagnosed DataLoader worker issues and suggested stable Colab execution practices.
Submission preparation	Produce report, README, tables, and repository structure	Reviewed the repository as an external evaluator and identified methodological and presentation inconsistencies.

4 Representative AI-Assisted Interactions

4.1 Understanding the Main Research Problem

Conversation 1: What does LwF solve?

Student question: What is the main idea of the paper, and what exactly is catastrophic forgetting?

AI guidance: Catastrophic forgetting occurs when the shared parameters of a neural network are modified for a new task and the model loses accuracy on tasks it previously performed well. LwF addresses this by using the new-task images twice: once with their ground-truth labels for learning the new task, and once as inputs to the frozen original model to obtain soft targets for preserving old-task behavior.

Action taken: The project was framed around two measurements after adaptation: performance on the new task and retained performance on ImageNet.

Impact on the project: This prevented the project from focusing only on CUB or Scenes accuracy. The old-task measurement became a central part of every experiment.

4.2 Clarifying When Old-Task Accuracy Is Measured

Conversation 2: Measuring ImageNet after learning the new task

Student question: Is ImageNet measured before training, after training, or both?

AI guidance: Measure ImageNet before adaptation to obtain a baseline, and measure it again after learning the new task. The difference between the two measurements quantifies forgetting. The post-training evaluation must use the retained ImageNet head of the adapted model, not the untouched original model.

Action taken: Separate evaluation functions were created for the original ImageNet baseline and the adapted models.

Impact on the project: This established a correct old-task evaluation protocol and later helped identify incorrect Fine-Tuning measurements.

4.3 Preparing the CUB Dataset

Conversation 3: Organizing CUB metadata

Student question: How should the CUB files be prepared for PyTorch?

AI guidance: Merge the metadata files `images.txt`, `image_class_labels.txt`, and `train_test_split.txt`. The resulting table should contain the image path, class label, and official train/test assignment. Then build a custom Dataset class and DataLoader objects.

Action taken: The CUB notebook was divided into explicit preparation stages: metadata loading, merging, Dataset creation, DataLoader creation, and verification.

Impact on the project: The data pipeline became transparent and reproducible rather than relying on manually arranged image folders.

4.4 Designing the Dual-Head LwF Model

Conversation 4: Teacher, student, and task-specific heads

Student question: Why does LwF need two heads, and which model should be frozen?
AI guidance: The teacher is the original ImageNet-pretrained AlexNet and remains frozen. The student contains shared AlexNet layers, an old ImageNet head with 1,000 outputs, and a new-task head with 200 outputs for CUB or 67 outputs for Scenes. During LwF training, new-task labels supervise the new head and teacher logits supervise the old head through distillation.
Action taken: Ambiguous variables such as `old_model` were replaced by clearer names such as `pretrained_model`, `teacher_model`, and `student_model`.
Impact on the project: The model roles became easier to verify and accidental evaluation of the wrong model became less likely.

4.5 Understanding Temperature and Lambda

Conversation 5: Why use $T = 2$ and $\lambda_{\text{old}} = 1$?

Student question: What does the temperature parameter do? How does changing lambda affect the model?
AI guidance: Temperature softens the teacher and student output distributions:

$$q_i = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)}.$$

A temperature above one reveals information about similarities between classes. The paper uses $T = 2$ based on validation. The coefficient λ_{old} controls the balance between preserving old knowledge and adapting to the new task. Larger values prioritize retention; smaller values permit stronger adaptation.
Action taken: The implementation used `TEMPERATURE = 2.0` and `LAMBDA_OLD = 1.0`. Adaptive lambda scheduling was proposed as the bonus improvement.
Impact on the project: The loss function was implemented with a clear interpretation rather than treated as a black box.

4.6 Debugging Suspiciously Identical Baseline and LwF Results

Challenge 1: LwF ImageNet accuracy looked identical to the baseline

Student question: The LwF ImageNet accuracy is almost the same as the original baseline. Is that expected, or is there a bug?
AI guidance: A nearly identical result may be real, but it can also indicate an evaluation mistake. Inspect the following possibilities:

- evaluating the teacher instead of the student,
- reloading the original AlexNet before evaluation,

- leaving shared layers frozen during LwF training,
- using the new-task head instead of the old-task head,
- saving or loading the wrong checkpoint,
- excluding trainable shared parameters from the optimizer.

Action taken: The evaluation function was rewritten to explicitly read the student's ImageNet logits. The optimizer and freezing logic were inspected. The implementation uses a dual-head student and a frozen teacher.
Impact on the project: This was a major debugging step. It prevented a misleading old-task result and made the ImageNet retention measurement trustworthy.

4.7 Correcting Fine-Tuning Old-Task Evaluation

Challenge 2: Fine-Tuning ImageNet accuracy was not measured correctly

Student question: If the Fine-Tuning classifier head is replaced for the new task, should ImageNet accuracy simply be reported as zero?
AI guidance: No. Replacing the active prediction head does not prove that ImageNet accuracy is zero. To measure forgetting, retain the original ImageNet head, keep it fixed during Fine-Tuning, adapt the shared representation and the new-task head, and then evaluate ImageNet through the retained old head.
Action taken: Fine-Tuning was implemented with a dual-head model. The shared representation and new head are trained, while the old ImageNet head is retained and frozen for evaluation.
Impact on the project: The Fine-Tuning comparison became experimentally valid. Old-task degradation is now measured rather than inserted manually.

4.8 Preparing the MIT Indoor Scenes Dataset

Conversation 6: Building the Scenes dataset pipeline

Student question: How should the MIT Indoor Scenes files be organized so that the notebook can train and evaluate the model correctly?
AI guidance: Use the official image lists to preserve the dataset split. Read the training-image list and the test-image list, extract the scene-category name from each relative path, build one shared class-to-index mapping, and use the same mapping for training, validation, and testing. The dataset contains 67 indoor-scene categories, so the new classification head must produce 67 logits.
Action taken: A dedicated Scenes Dataset class was created. It reads image paths and labels from the official lists, opens each RGB image, applies the selected transform, and returns the image tensor together with its integer class index.
Impact on the project: The Scenes pipeline became explicit and reproducible. Category

labels are generated consistently across all loaders rather than inferred separately for each split.

4.9 Creating the Scenes Training, Validation, and Test Loaders

Conversation 7: Preserving the official test set

Student question: How should the Scenes loaders be divided so that the test set is not used during training?

AI guidance: Keep the official Scenes test set untouched until final evaluation. Split only the official training images into an internal training subset and a held-out validation subset. Use data augmentation for the training loader and deterministic preprocessing for validation and testing. The validation loader should be used to select the best checkpoint.

Action taken: The notebook creates three loaders: a shuffled training loader, a deterministic validation loader, and a deterministic official-test loader. The same class mapping is passed to all three datasets.

Impact on the project: The Scenes evaluation reflects generalization to the official test set. Model selection is based on validation performance rather than repeated inspection of test accuracy.

4.10 Building the AlexNet Heads for Scenes

Conversation 8: Adapting the dual-head model to 67 scene classes

Student question: How should the CUB model structure be adapted for the Scenes experiment?

AI guidance: Keep the same ImageNet-pretrained AlexNet representation and the same 1,000-output ImageNet head. For the new task, create a separate head with 67 outputs. Fine-Tuning should train the shared representation and the Scenes head while retaining the frozen ImageNet head for old-task evaluation. LwF should use a frozen ImageNet teacher and a dual-head student. Feature Extraction should keep the pretrained representation frozen and train only the scene classifier.

Action taken: The Scenes notebook uses an ImageNet teacher model, a dual-head student model with 1,000 ImageNet outputs and 67 Scenes outputs, and separate model constructors for Fine-Tuning and Feature Extraction.

Impact on the project: The three methods are compared on the same task pair with clearly separated old-task and new-task outputs.

4.11 Defining the Scenes Training and Evaluation Flow

Conversation 9: Running all compared methods in a consistent order

Student question: What sequence should the Scenes notebook follow from baseline measurement to the final table?

AI guidance: First measure the pretrained ImageNet baseline. Next warm up the new Scenes head. Then train and evaluate Fine-Tuning, Feature Extraction, and LwF separately. For every method, measure the complete official Scenes test set and evaluate ImageNet through the appropriate old-task head on the same fixed 80-batch subset. Finally, place all measurements in one results table.

Action taken: The notebook was organized into explicit stages: environment setup, dataset preparation, ImageNet baseline evaluation, warm-up, Fine-Tuning, Feature Extraction, LwF, and final reporting.

Impact on the project: The code structure makes the full Scenes experiment easy to follow, rerun, and review cell by cell.

4.12 Using Warm-Up and Validation-Based Checkpoint Selection

Conversation 10: Defining the training protocol

Student question: What should be added to make the reproduction more faithful and reduce overfitting?

AI guidance: Add a warm-up stage in which only the newly initialized head is trained while the pretrained representation remains frozen. Then use the warmed-up model as the common starting point for Fine-Tuning and LwF. Also split the official training set into training and validation subsets, select the best checkpoint by validation accuracy, and reserve the official test set for final evaluation only.

Action taken: Both notebooks now use one warm-up epoch, five main training epochs, internal validation, checkpoint selection by validation accuracy, and official test evaluation only after training.

Impact on the project: The methodology became more defensible and closer to the paper's experimental logic.

4.13 Implementing the Feature Extraction Baseline

Conversation 11: Why did Feature Extraction perform strongly?

Student question: Feature Extraction achieved the strongest result in one Scenes run. Is that suspicious?

AI guidance: It can happen because freezing the representation acts as strong regularization, especially with a small dataset. However, the implementation should still match the intended baseline: disable dropout inside the frozen extractor and train a two-layer classifier with a 4,096-unit hidden layer. Also add data augmentation and choose checkpoints by validation accuracy.

Action taken: Feature Extraction was implemented with a frozen representation, disabled dropout in the frozen part, and a two-layer head:

```
4096 -> 4096 -> number of new classes
```

Impact on the project: The baseline became closer to the paper and the comparison became more meaningful.

4.14 Keeping ImageNet Evaluation Consistent

Conversation 12: Why exactly 80 ImageNet batches?

Student question: Should the new code evaluate a random subset of ImageNet, or should it keep the original 80-batch approach?

AI guidance: A random subset could be scientifically reasonable, but consistency across CUB, Scenes, the README, the report, and the lecturer-approved protocol is more important for this submission. Evaluate the first 80 ImageNet validation batches with `batch_size=64` and `shuffle=False` for every method.

Action taken: Both notebooks use:

```
IMAGENET_NUM_BATCHES = 80
BATCH_SIZE_IMAGENET = 64
shuffle = False
```

Impact on the project: All old-task comparisons are based on the same 5,120 ImageNet validation images.

4.15 Clarifying the Meaning of a Seed

Conversation 13: One run with seed 42

Student question: Why does the notebook print "starting run with seed 42"? Does that mean 42 runs?

AI guidance: No. A seed is a number that fixes random decisions such as parameter initialization, data order, train-validation splitting, and augmentation. The configuration `RUN_SEEDS = [42]` means exactly one run.

Action taken: Both notebooks were configured for one complete run:

```
RUN_SEEDS = [42]
```

Impact on the project: The report states that the reproduction uses one fixed seed due to computational constraints.

4.16 Handling a Colab Runtime Stall

Challenge 4: The warm-up stage appeared stuck

Student question: The notebook has displayed the warm-up heading for more than half an hour without printing an epoch result. Is this normal?

AI guidance: If no epoch result appears after a long delay, the process may be stuck while loading data. In Colab, DataLoader worker processes can occasionally stall, especially with Google Drive files or after interrupting a run. A more stable fallback is `NUM_WORKERS = 0`. Restarting the session and running the notebook sequentially also prevents stale variables and worker processes from affecting the execution.

Action taken: The execution procedure was clarified: restart the session when necessary, run cells sequentially, and use a stable DataLoader configuration if the stall repeats.

Impact on the project: The notebook completed successfully after restarting execution. The issue was treated as a runtime stability problem rather than incorrectly modifying the learning algorithm.

4.17 Interpreting the Final Results

Conversation 14: Do the final results reproduce the paper's trend?

Student question: The numerical values are not identical to the paper. Are the results still acceptable?

AI guidance: Exact equality is not expected because the reproduction uses a limited ImageNet subset, one seed, limited training time, and a PyTorch implementation. The important question is whether the qualitative pattern is reproduced:

- Feature Extraction preserves ImageNet best because its representation is frozen.
- Fine-Tuning causes the strongest ImageNet degradation.
- LwF preserves substantially more old-task knowledge than Fine-Tuning while learning the new task effectively.

Action taken: The report presents both absolute results and differences relative to LwF, following the table style of the original paper.

Impact on the project: The final discussion focuses on scientifically defensible trends rather than forcing the results to match the paper numerically.

4.18 Preparing the Repository and Submission Files

Conversation 15: External repository review

Student question: The lecturer plans to upload the GitHub repository to an AI evaluator. Can you review the repository as if you were the evaluator?

AI guidance: Review the README, notebook structure, architecture consistency, saved outputs, evaluation logic, security hygiene, result tables, and documentation. Verify that the repository can be understood without an oral explanation and that no notebook is saved with tracebacks or hard-coded external-service credentials.

Action taken: The repository structure, README, report, result tables, notebook outputs, and AI documentation were revised to match the experimental protocol.

Impact on the project: The submission became more coherent and easier to audit automatically.

5 Challenge Resolution Matrix

Challenge	Why It Mattered	Resolution	Result
Unclear old-task evaluation	Could lead to meaningless ImageNet numbers	Evaluate adapted models through the retained ImageNet head	Forgetting is measured directly after new-task training
Identical baseline and LwF values	Could indicate teacher/student confusion	Inspect checkpoint loading, frozen layers, optimizer parameters, and output-head selection	LwF old-task evaluation is explicitly tied to the student's old head
Fine-Tuning ImageNet value inserted manually	A manual value is not an experimental result	Preserve the old head and measure ImageNet after adaptation	Fine-Tuning degradation is now measured correctly
Scenes label mapping and split	Separate label mappings or test leakage would invalidate the experiment	Build one shared 67-class mapping and reserve the official test set for final evaluation	Reliable Scenes training and evaluation pipeline
Overfitting risk	Small new-task datasets can produce unstable test results	Add augmentation, held-out validation, and best-checkpoint selection	More defensible new-task evaluation
Protocol inconsistency	Different ImageNet subsets would weaken comparisons	Use the first 80 batches for every method and task pair	Consistent old-task evaluation across the project
Seed confusion	Multiple runs would multiply runtime and complicate reporting	Use one fixed seed: 42	Reproducible single-run protocol

Challenge	Why It Mattered	Resolution	Result
Colab stall	Runtime appeared frozen before the first epoch	Restart execution and use a stable DataLoader fallback when needed	Successful completion of both notebooks
Repository presentation	An AI evaluator must understand the project from GitHub alone	Align README, report, tables, notebook names, and outputs	Auditable final submission

6 Final Experimental Protocol

The CUB and Scenes notebooks use the same protocol.

Table 3: Experimental protocol used in both reproduced scenarios.

Setting	Value
Architecture	ImageNet-pretrained AlexNet
Number of runs	1
Random seed	42
Warm-up duration	1 epoch
Main training duration	5 epochs per method
CUB batch size	32
Scenes batch size	32
ImageNet evaluation batch size	64
ImageNet evaluation batches	80
ImageNet DataLoader shuffling	Disabled
Distillation temperature	$T = 2$
Old-task loss coefficient	$\lambda_{\text{old}} = 1$
Optimizer	SGD
Momentum	0.9
Weight decay	0.0005
Checkpoint selection	Best internal validation accuracy
Official test sets	Used only for final evaluation

7 Final Experimental Results

7.1 Absolute Performance

Table 4: Absolute performance measured in the reproduced experiments.

Experiment	Method	Old: ImageNet	New Task
ImageNet → CUB	LwF	59.49	45.41
ImageNet → CUB	Fine-Tuning	53.42	45.53
ImageNet → CUB	Feature Extraction	62.89	42.03
ImageNet → Scenes	LwF	59.82	56.27
ImageNet → Scenes	Fine-Tuning	48.69	50.82
ImageNet → Scenes	Feature Extraction	62.89	55.00

7.2 Differences Relative to LwF

Following the presentation style of the original paper, the LwF row reports absolute performance while the other rows report their difference relative to LwF.

Table 5: Performance differences relative to LwF.

Experiment	Method	Old: ImageNet	New Task
ImageNet → CUB	LwF (ours)	59.49	45.41
ImageNet → CUB	Fine-Tuning	−6.07	+0.12
ImageNet → CUB	Feature Extraction	+3.40	−3.38
ImageNet → Scenes	LwF (ours)	59.82	56.27
ImageNet → Scenes	Fine-Tuning	−11.13	−5.45
ImageNet → Scenes	Feature Extraction	+3.07	−1.27

The results reproduce the main qualitative behavior described in the paper. Feature Extraction preserves ImageNet most effectively because the representation remains frozen. Fine-Tuning causes the largest decrease in old-task performance. LwF provides a compromise: it retains substantially more ImageNet knowledge than Fine-Tuning while learning the new task effectively.

In the CUB experiment, Fine-Tuning exceeds LwF on the new task by only 0.12 percentage points. With one seed, this difference is too small to support a strong conclusion. The old-task retention advantage of LwF remains clear.

8 Division of Responsibility: Students and AI Assistant

8.1 Work Performed by the Students

- Selecting the paper and defining the reproduction scope.
- Downloading, arranging, and verifying the datasets.
- Verifying the official CUB metadata split and the official Scenes image lists.

- Running all notebooks in Google Colab.
- Managing Google Drive paths and Colab resources.
- Collecting numerical results from completed runs.
- Reviewing the final interpretation and deciding which files to submit.
- Uploading the repository and submission files.

8.2 Work Supported by the AI Assistant

- Explaining the theory of LwF and distillation.
- Suggesting a clear notebook workflow.
- Guiding the construction of the CUB metadata pipeline and the Scenes image-list pipeline.
- Reviewing code logic and identifying evaluation mistakes.
- Recommending consistent experimental settings.
- Explaining unexpected results and possible causes.
- Helping troubleshoot runtime issues.
- Drafting report, README, and result-table text for student review.

The AI assistant did not execute the final experiments on the students' datasets. The numerical values were obtained from notebook runs performed by the students.

9 Lessons Learned

The project demonstrated that successful reproduction requires more than implementing a loss function. Several small details can substantially affect the validity of the conclusions:

- The model used for evaluation must be explicitly verified.
- Old-task accuracy must be measured after adaptation, not inferred.
- A frozen baseline and an adaptable method must be compared under the same evaluation subset.
- Architecture choices must match the stated experimental scope.
- A shared class mapping must be used consistently across the Scenes training, validation, and test loaders.
- Validation-based checkpoint selection is important when the new datasets are relatively small.

- Reproducibility settings, such as random seeds and batch limits, should be stated clearly in the report.
- Notebook outputs, README text, and final tables must all describe the same experiment.

10 Conclusion

The AI-assisted interactions documented here show the complete progression of the project: understanding the paper, constructing the CUB and Scenes data pipelines, implementing the baselines, building the LwF teacher-student architecture, debugging suspicious measurements, aligning the two experiments, resolving runtime issues, interpreting the final results, and preparing the repository for evaluation.

The most valuable AI contribution was not simply code generation. It was the iterative review process: asking whether the measurements were valid, whether the model roles were clear, whether the notebooks were consistent with the paper, and whether the repository told a coherent technical story.

The experiments reproduce the core LwF trade-off under practical computational constraints. Fine-Tuning learns the new task but forgets more of ImageNet, Feature Extraction preserves ImageNet but adapts less strongly, and LwF provides a balanced continual-learning solution without access to the original ImageNet training data.

11 Reference

Li, Z., & Hoiem, D. (2016). *Learning without Forgetting.* European Conference on Computer Vision (ECCV), pp. 614–629.
https://arxiv.org/abs/1606.09282